Partie 4 : Natural Language Processing (NLP)

Lino Galiana
Version du 2 novembre 2026

Cette partie du cours introduit le traitement automatique du langage (NLP), un domaine scientifique à la croisée de la linguistique et des statistiques devenu, du fait de l'engouement envers les IA génératives, central dans le domaine de la *data science*. A travers des exemples littéraires, dcette partie explore d'abord des méthodes classiques comme l'analyse fréquentiste et le traitement de corpus textuels sous la forme de *bag of words*. Ensuite, elle aborde la modélisation du langage qui ouvre la voie à des approches plus originales. L'objectif de chapitre est de rappeler quelques éléments généraux sur le vaste domaine qu'est le NLP.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **1**
2 Le traitemen automatique du langage **2**
3 Résumé de la partie **2**
3.1 Nettoyages textuels et analyse de fréquences 3
3.2 Modélisation du langage 3
Pour aller plus loin **4**
Informations additionnelles 5

1 Introduction

Les parties précédentes étaient consacrées à l'acquisition de compétences transversales pour la valorisation des données. De manière naturelle, nous nous sommes jusqu'à présent plutôt consacrés à la valorisation de données structurées, d'une dimension certes modeste mais qui ouvraient déjà énormément de problématiques à creuser. Cette partie propose maintenant de se consacrer à un sujet dont il n'est pas évident *a priori* que les ordinateurs s'emparent, source de débats philosophiques séculaires, de Platon à Saussure : le **langage humain** et sa richesse.

En faisant l'analogie entre langue et langage, c'est-à-dire en définissant ce dernier comme la capacité d'expression et de communication d'une pensée par le biais de signes et en définissant la langue comme la mise en œuvre conventionnelle de cette capacité, on peut se placer dans les traces de la linguistique et représenter le langage sous une forme de données. Ceci ouvre la voie à l'analyse statistique ou algorithmique. Néanmoins, même s'il existe des régularités statistiques, comment des ordinateurs, qui au fond ne connaissent que le *0* et le *1*, peuvent-ils s'approprier cet objet éminemment complexe qu'est le langage et qu'un humain met lui-même des années à comprendre et s'approprier ?[1]

[1] En écrivant ces lignes j'ai demandé à `ChatGPT` de me faire une blague.

> Pourquoi les programmeurs détestent-ils la plage ? Parce que le sable change tout le temps de paradigme !

2 Le traitemen automatique du langage

Le traitement automatique du langage — traduction française du concept de *natural language processing* (NLP) — est l'ensemble des techniques permettant aux ordinateurs de comprendre, analyser, synthétiser et générer le langage humain[2].

Il s'agit d'un champ disciplinaire à l'intersection de la statistique et de la linguistique qui connaît depuis quelques années un engouement important, que ce soit d'un point de vue académique, opérationnel ou industriel. Certaines des applications de ces techniques sont devenues incontournables dans nos tâches quotidiennes, notamment les moteurs de recherche, la traduction automatique et plus récemment les *chatbots*, dont le développement connaît depuis l'émergence de `ChatGPT` en décembre 2022 un rythme fou.

3 Résumé de la partie

Cette partie du cours est consacrée à l'analyse des données textuelles avec des exemples de 📖 pour s'amuser. Elle est une introduction progressive à ce sujet en se concentrant sur des concepts de base, nécessaires à la compréhension ultérieure de principes plus avancés et de techniques sophistiquées[3]. Cette partie présente principalement :

- Les enjeux de nettoyage de champs textuels et d'analyse de fréquence. Il s'agit de NLP un peu *old school* mais dont la compréhension est nécessaire pour aller plus loin ;
- La modélisation du langage, selon plusieurs approches.

Avant d'en arriver au sujet des *embeddings*, il est précieux de comprendre les apports et les limites de concepts comme le sac de mot (*bag of words*) ou la distance TF-IDF (*term frequency - inverse document frequency*). L'un des apports principaux des grands modèles de langage, à savoir la richesse de la fênetre contextuelle leur permettant de mieux saisir les nuances textuelles et l'intentionalité du locuteur, s'éclairent lorsqu'on saisit les limites du NLP traditionnel.

Ne l'ayant pas comprise, je lui ai demandé de me l'expliquer. Voici sa réponse :

> Bien sûr, je serais ravi de l'expliquer ! Cette blague joue sur le double sens du mot « paradigme » et fait un jeu de mots avec les notions liées à la programmation et à la plage.
>
> Dans le domaine de la programmation, un « paradigme » désigne un modèle ou une méthode de programmation, comme la programmation orientée objet, la programmation fonctionnelle, etc. Les programmeurs sont souvent familiers avec différents paradigmes de programmation.
>
> D'autre part, en contexte de plage, « paradigme » est utilisé pour signifier « marée qui monte et descend », se référant à l'action naturelle du sable et de l'eau sur la plage.
>
> La blague joue sur la frustration humoristique que peuvent ressentir les programmeurs lorsqu'ils doivent s'adapter à de nouveaux paradigmes de programmation, tout comme le sable de la plage qui change constamment avec les marées. En associant les deux sens du mot « paradigme », la blague crée une situation absurde et amusante, faisant sourire ceux qui comprennent les références à la programmation et à la plage.

Finalement, j'ai trouvé son explication plus drôle que sa blague.

[2]Le concept de *natural language processing* tend à remplacer celui de *text mining* dont le périmètre était plus spécifique. Le *text mining* renvoie à la recherche d'information — la fouille — dans un ensemble de textes. C'est l'un des champs d'applications du *natural language processing* mais ce n'est pas l'unique. L'accroissement des ressources de calcul et les progrès dans la formalisation du langage ont permis d'élargir le champ des domaines où la linguistique computationnelle intervient.

[3]Par exemple, le concept d'*embedding* - transformation d'un champ textuel en un vecteur numérique multidimensionnel - aujourd'hui central dans le NLP n'est évoqué qu'à quelques reprises.

Dans une optique introductive, ce cours se focalise donc sur les approches fréquentistes, notamment l'approche sac de mot, pour faciliter l'ouverture ultérieure de la boite de Pandore que sont les *embeddings*.

3.1 Nettoyages textuels et analyse de fréquences

`Python` est un excellent outil pour l'analyse de données textuelles. Les méthodes de base de transformation de données textuelles ou de dictionnaires, associées à des librairies spécialisées comme `NLTK` et `SpaCy`, permettent d'effectuer des tâches de normalisation et d'analyse de données textuelles de manière très efficace. `Python` est bien mieux outillé que `R` pour l'analyse de données textuelles. Les ressources en ligne sur le sujet sont très nombreuses et la meilleure des écoles dans le domaine reste la pratique sur un corpus à nettoyer.

Dans un premier temps, cette partie propose de revenir sur la manière de structurer et nettoyer un corpus textuel au travers de l'approche *bag of words* (sac de mots). Elle vise à montrer comment transformer un corpus en outil propre à une analyse statistique :

- Elle propose d'abord une introduction aux enjeux du nettoyage des données textuelles à travers l'analyse du *Comte de Monte Cristo* d'Alexandre Dumas ici qui permet de synthétiser rapidement l'information disponible dans un large volume de données (à l'image des *wordcloud*)
- Elle propose ensuite une série d'exercices sur le nettoyage de textes à partir des oeuvres d'Edgar Allan Poe, Mary Shelley et H.P. Lovecraft visant à distinguer la spécificité du vocabulaire employé par chaque auteurs. Ces exercices sont disponibles dans le deuxième chapitre de la partie.

Cette analyse fréquentiste permet de prendre du recul sur la nature des données textuelles et sur les enjeux récurrents dans la réduction de dimension de corpus en langue naturelle. Comme la statistique descriptive entraîne naturellement la modélisation, cette approche fréquentiste va généralement amener rapidement à vouloir synthétiser quelques lois derrière nos corpus textuels.

3.2 Modélisation du langage

La suite de cette partie proposera une introduction aux enjeux de modélisation du langage. Ceux-ci sont très à la mode du fait du succès de `ChatGPT`. Néanmoins, avant d'en arriver aux grands modèles de langage (LLM), ces réseaux de neurone ayant des milliards de paramètres et entraînés sur des volumes massifs de données, il est nécessaire de passer par quelques modélisations préliminaires.

Nous proposerons d'abord d'explorer une approche alternative, prenant en compte le contexte d'apparition d'un mot. L'introduction à la *Latent Dirichlet Allocation* (LDA) sera l'occasion de présenter la modélisation de documents sous la forme de *topics*. Celle-ci est néanmoins passée de mode au profit des méthodes liées au concept d'*embedding*.

Nous introduirons ainsi à la fin de cette partie du cours les enjeux de la transformation de champs textuels sous forme de vecteurs numériques. Pour cela, nous présenterons le principe de `Word2Vec` qui permet ainsi, par exemple, malgré une distance syntaxique importante, de dire que sémantiquement `Homme` et `Femme` sont proches. Ce chapitre est une passerelle vers le concept d'*embedding*, véritable révolution récente du NLP, et qui permet de rapprocher des corpus non seulement sur leur proximité syntaxique (partagent-ils par exemple des mots communs ?) mais aussi sur leur proximité sémantique (partagent-ils un thème ou un sens commun ?). Ce passage par `Word2Vec` permettra aux curieux de pouvoir ensuite passer aux modèles de type *transformers*, les modèles faisant aujourd'hui office de référence dans le domaine du NLP.

Pour aller plus loin

La recherche dans le domaine du NLP est très active. Il est donc recommandé de faire preuve de curiosité pour en apprendre plus car une ressource unique ne compilera pas l'ensemble des connaissances, *a fortiori* dans un champ de recherche aussi dynamique que le NLP.

Pour approfondir les compétences évoquées dans ce cours, je recommande vivement ce cours d'`HuggingFace`.

Pour comprendre l'architecture interne d'un LLM, ce post de Sebastian Raschka est très utile.

Ces chapitres n'épuisent pas les cas d'usage du NLP pour les *data scientists*. Ils n'en sont que la surface émergée de l'iceberg. Par exemple, dans le domaine de la statistique publique, un des principaux cas d'usage du NLP est l'utilisation de techniques de classification automatique pour transformer des réponses libres dans des questionnaires en champs prédéfinis dans une nomenclature. Il s'agit donc d'une adaptation, un peu spécifique à la statistique publique, grande utilisatrice de nomenclatures normalisées, de problématiques de classification multi-niveaux.

Voici un exemple sur un projet de classification automatisée des professions dans la typologie des nomenclatures d'activités (les PCS) à partir d'un modèle entraîné par la librairie `Fasttext` :

```
import requests
import pandas as pd

activite = "data scientist"
urlApe = (
    "https://codification-ape-test.lab.sspcloud.fr/"
    f"predict?nb_echos_max=3&prob_min=0&text_feature={activite}"
)

try:
    # requête
    resp = requests.get(urlApe, timeout=10)
    resp.raise_for_status()  # lève une erreur si code HTTP ≠ 200
    data = resp.json()

    # récupération de IC
    IC = data.pop("IC", None)

    # transformation en DataFrame
    df = pd.DataFrame(data.values())
    df["indice_confiance"] = IC

    print(df)

except requests.exceptions.RequestException as e:
    print("Erreur lors de l'appel API :", e)
    df = pd.DataFrame()  # DataFrame vide en cas d'échec

except (ValueError, KeyError) as e:
    print("Erreur lors du parsing des données :", e)
    df = pd.DataFrame()
```

```
Erreur lors de l'appel API : 503 Server Error: Service Unavailable for url: https://codification-ape-test.lab.sspcloud.fr/predict?nb_echos_max=3&prob_min=0&text_feature=data%20scientist
```

Informations additionnelles

Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "py7zr≥1.1.3",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
    "python-dotenv≥1.0.1",
    "python-frontmatter≥1.1.0",
    "pywaffle≥1.1.1",
    "requests≥2.32.3",
    "scikit-image≥0.24.0",
    "scikit-learn≥1.8.0",
    "scipy≥1.13.0",
    "seaborn≥0.13.2",
    "selenium<4.39.0",
    "spacy≥3.8.4",
    "webdriver-manager≥4.0.2",
```

```
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.